CRIMEVISION AI

Multimodal Crime Scene Analysis and Forensic Intelligence Platform

PROJECT DOCUMENTATION

Document Purpose

This document describes CrimeVision AI using the milestone and activity list provided. It explains the project purpose, representative scenarios, technical architecture, prerequisites, implementation workflow, and user-facing website experience. Dedicated screenshot spaces are included throughout.

Accuracy notes: the supplied screenshots show the milestone plan and prerequisites, not the running application. The documentation therefore follows the stated scope and named technologies. Verify exact page names, routes, model versions, input fields, test results, and deployment details against the final code before submission.

Table of Contents

1. Project Title
2. Project Description
3. Project Scenarios (1–4)
4. Technical Architecture
5. Prerequisites
6. Project Workflow — overview of milestones and activities
7. Exploring the Website: pages, inputs, and results
8. Conclusion

1. Project Title

CrimeVision AI: Multimodal Crime Scene Analysis and Forensic Intelligence Platform

CrimeVision AI is an AI-assisted web application intended to analyze image, audio, and video evidence through separate processing modules and present their outputs in a unified review interface. The planned capabilities include object detection, image captioning, speech transcription, video-frame analysis, event extraction, timeline generation, natural-language querying, and report generation.

2. Project Description

CrimeVision AI is an AI-powered web application designed to assist in crime scene investigation by analyzing images, audio recordings, and video footage. The system uses technologies such as Python, Flask, OpenCV, PyTorch, Whisper, and large language models to detect objects, transcribe audio, analyse video frames, and extract relevant events.

The application combines these analysis results into a unified forensic timeline, enabling users to review evidence, explore potential scenarios, and generate structured forensic reports through an interactive dashboard.

The primary objective of CrimeVision AI is to simplify digital evidence analysis, reduce manual effort, and support investigators in organizing and interpreting forensic information efficiently.

Field	**Description**
Project type	AI-assisted multimodal evidence-analysis web application
Backend	Python and Flask API
Image/video processing	OpenCV and PyTorch inference
Object detection	RT-DETR or the detector configured in the final implementation
Image captioning	BLIP, GIT, or a Florence alternative, depending on actual configuration
Audio transcription	Whisper
Language-model service	Groq LLM API, as listed in the prerequisites
Visualization	Three.js for 3D visualization, if implemented

3. Project Scenarios (1, 2, 3, and 4)

These four representative scenarios are derived from the supplied project scope. Replace example inputs and expected outputs with the exact test cases used in the completed application.

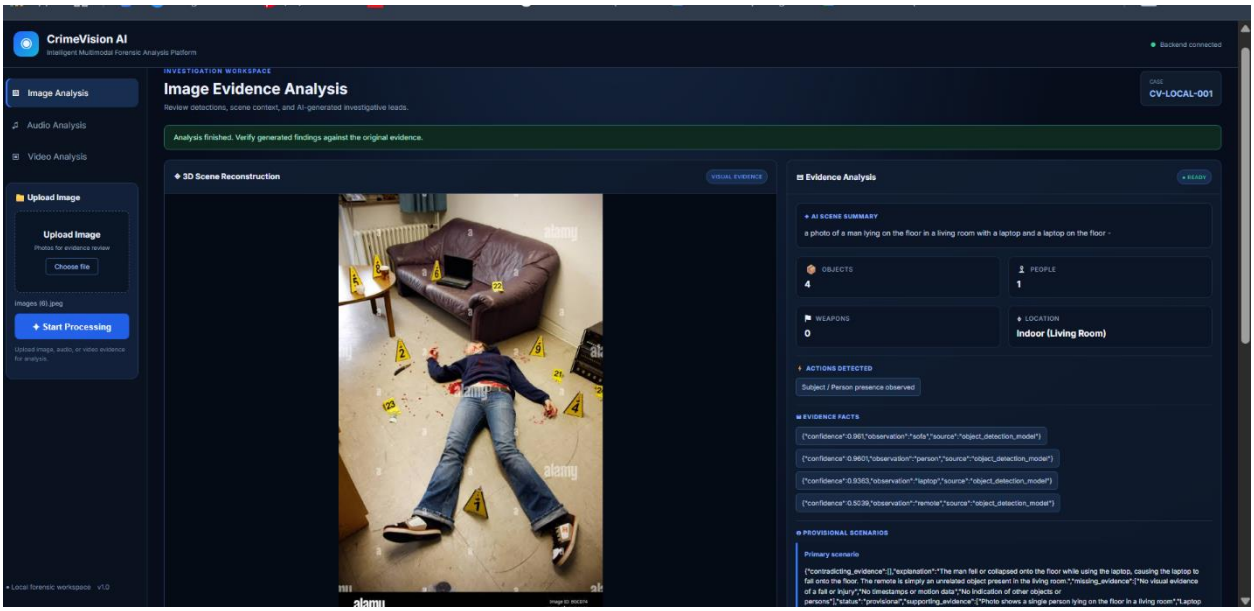
Scenario 1: Image Evidence Analysis

A user uploads a photograph for visual inspection.

Procedure

1. Open the image-analysis page and choose a supported image.
2. Submit the image to the analysis pipeline.
3. Review detected objects, confidence values if shown, and the generated caption.
4. Compare any tentative hypothesis with the original image.

Expected result: The interface displays the source image and structured visual findings. Exact classes and fields depend on the configured model.



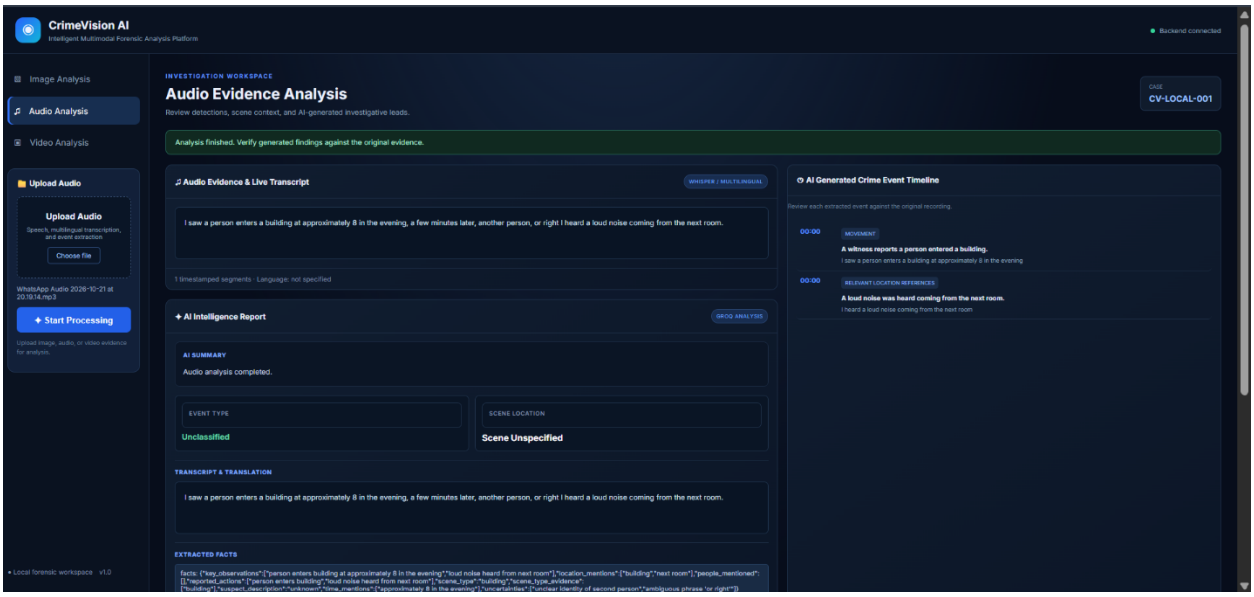
Scenario 2: Audio / Witness Statement Analysis

A user uploads an audio recording containing a witness statement or relevant conversation.

Procedure

1. Open the audio-analysis page and upload a supported recording.
2. Run speech transcription.
3. Review the transcript and timestamps, if available.
4. Inspect candidate events extracted from the transcript.

Expected result: The output is a transcript and, where implemented, a structured event list or timeline. Transcription should be checked against the recording.



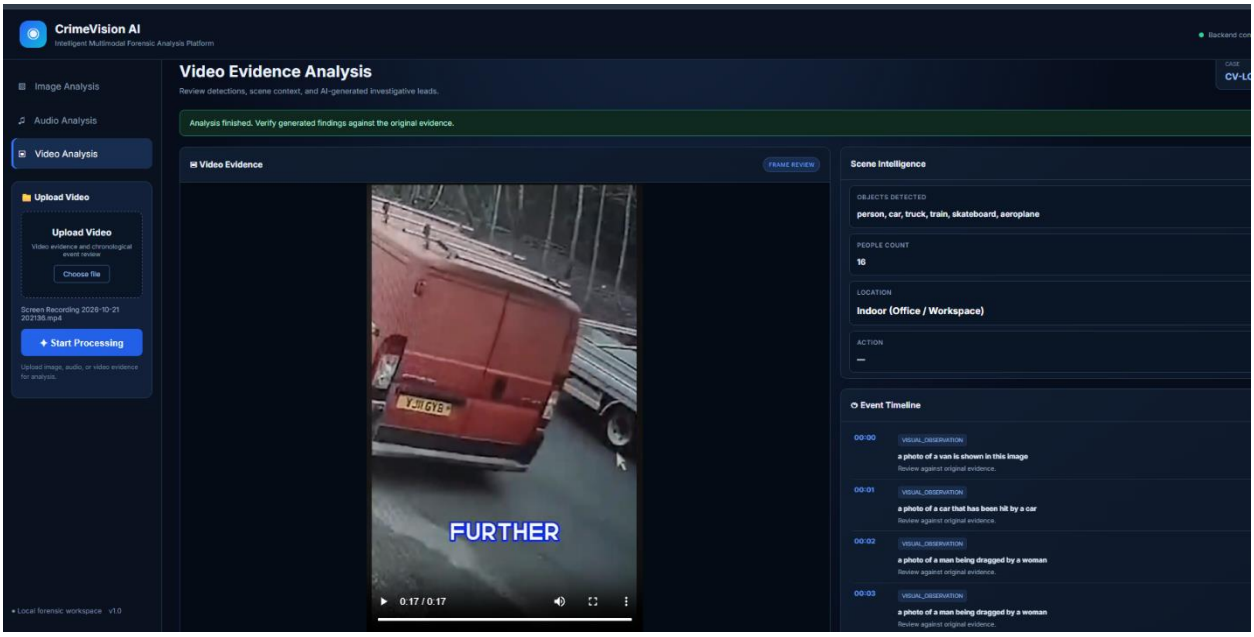
Scenario 3: Video Analysis and Event Timeline

A user uploads footage that may contain multiple events over time.

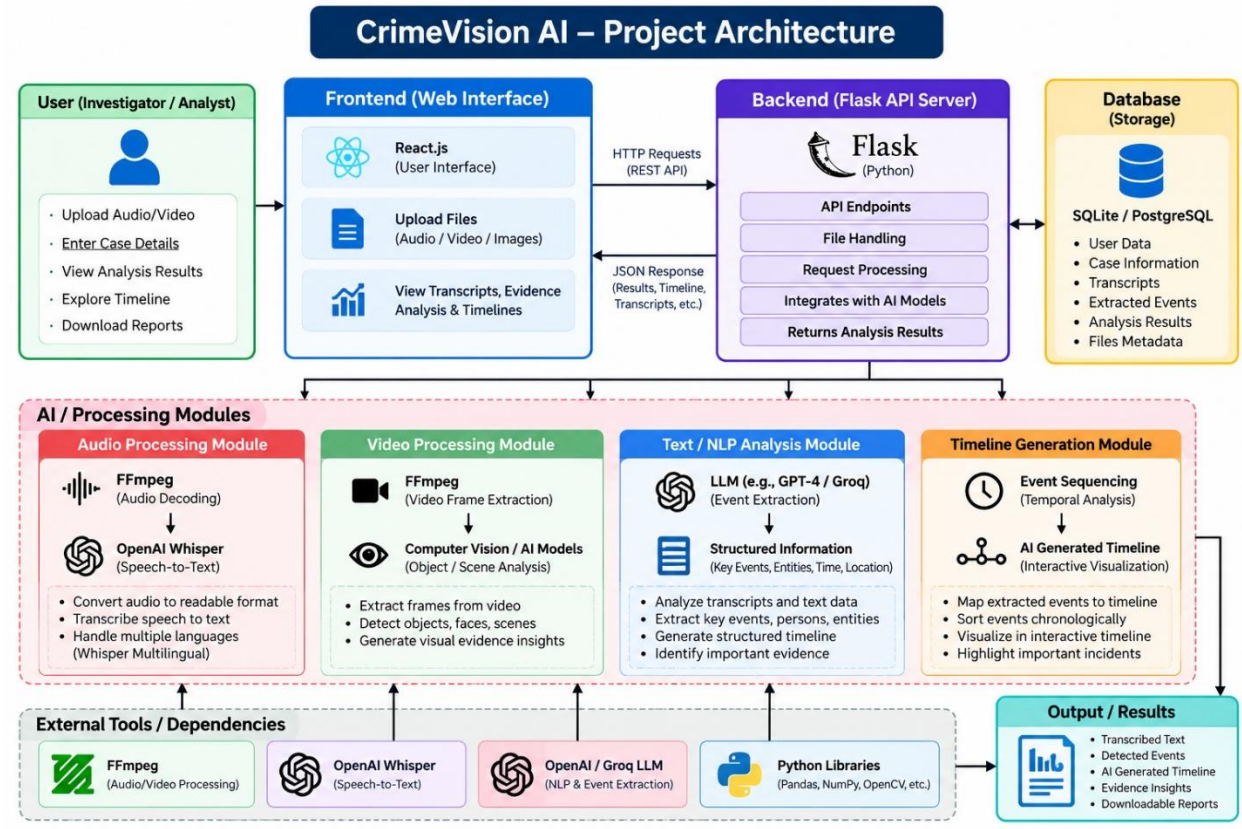
Procedure

1. Upload a supported video file.
2. Extract or sample frames for visual processing.
3. Run object detection and any configured action-recognition model.
4. Review timestamped observations and the event timeline.

Expected result: The output is a chronological list of observations. If frames are sampled, unsampled intervals may contain events the system did not detect.



4. Technical Architecture



5. Prerequisites

Requirement	Purpose / notes
Python 3.9+ (as listed)	**Backend and analysis scripts; match the version constraints in the project.**
pip and virtual environment	**Install dependencies in an isolated environment.**
Flask	**Backend API and request routing.**
OpenCV	**Image reading and video-frame processing; codec dependencies may apply.**
PyTorch	**Model inference; hardware needs depend on selected model and performance targets.**
RT-DETR / configured detector	**Object detection; confirm model checkpoint and supported classes.**
BLIP / GIT / Florence alternative	**Image captioning; include the model actually configured.**
Whisper	**Audio transcription; FFmpeg may be needed depending on setup.**
Groq API key	**External LLM service; requires network access and secure key storage.**
Three.js	**Only required if 3D visualization is implemented.**
Visual Studio Code	**Development editor; another compatible editor can also be used.**
Sample media	**Authorized test images, audio recordings, and video files.**

5.1 Environment Setup

1. Install the Python version supported by the project.
2. Create and activate a virtual environment.

3. Install the dependencies listed in requirements.txt.
4. Set required environment variables securely.
5. Start the Flask application using the repository's actual entry point.
6. Open the local application in a supported browser and test with a small, non-sensitive file.

Example commands — adapt these to the actual repository and operating system:

- python -m venv .venv
- Windows: .venv\Scripts\activate
- macOS/Linux: source .venv/bin/activate
- python -m pip install -r requirements.txt
- python app.py (only if app.py is the actual entry point)

6. Project Workflow — Milestones and Activities

The workflow moves from environment preparation to individual modality pipelines, multimodal fusion, and final interface validation. This overview lists milestone names and activities first; detailed explanations follow under separate milestone headings.

Milestone 1: Project Initialization and Environment Configuration

- **Activity 1.1:** Create requirements.txt.
- **Activity 1.2:** Install required libraries and set up a virtual environment.

Milestone 2: Image Analysis Module Development

- **Activity 2.1**: Implement object detection for people, weapons, and evidence using the video/image analysis pipeline.
- **Activity 2.2:** Generate scene descriptions and possible scenario hypotheses from image inputs.

Milestone 3: Audio Analysis Module Development

- **Activity 3.1:** Implement audio transcription for witness statements.
- **Activity 3.2:** Extract key events and generate structured timelines**.**

Milestone 4: Video Analysis and Timeline Generation

- **Activity 4.1:** Process video frames for object detection and action recognition.
- **Activity 4.2:** Generate second-by-second event timelines and crime type classification.

Milestone 5: Multimodal Fusion and Intelligence Generation

- **Activity 5.1:** Fuse image, audio, and video outputs into a unified forensic timeline.
- **Activity 5.2:** Implement natural-language scene querying and hypothesis generation using stored forensic context.

Milestone 6: Interface, Testing, and Deployment

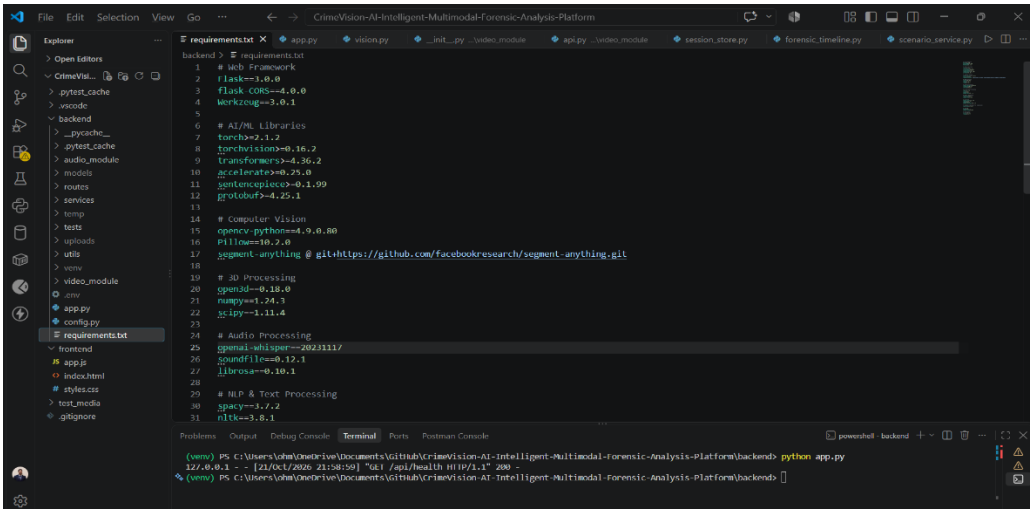
- **Activity 6.1:** Integrate all analysis modules into the CrimeVision AI dashboard.
- **Activity 6.2:** Validate system outputs using unit and integration tests.
- **Activity 6.3:** Generate forensic reports and finalize application deployment.

6A. Milestone 1: Project Initialization and Environment Configuration

Milestone objective: implement the described capability and capture evidence of its behavior in the actual application.

Activity 1.1: Create requirements.txt

The dependency manifest records the packages needed to run the application and makes setup repeatable. Review imports and integrations, list direct dependencies with tested versions where known, avoid unused packages, and verify installation in a clean environment.



Activity 1.2: Install required libraries and set up a virtual environment

A virtual environment isolates this project's packages from other Python projects. Install the supported Python version, create and activate the environment, install the dependency list, configure secrets as environment variables, and run an import/startup check. Document system dependencies such as codecs if required.

```
    + FullyQualifiedErrorId : CouldNotAutoLoadModule

PS C:\Users\ohm\OneDrive\Documents\GitHub\CrimeVision-AI-Intelligent-Multimodal-Forensic-Analysis-Platform\backend> .\venv\Scripts\activate
(venv) PS C:\Users\ohm\OneDrive\Documents\GitHub\CrimeVision-AI-Intelligent-Multimodal-Forensic-Analysis-Platform\backend> python app.py

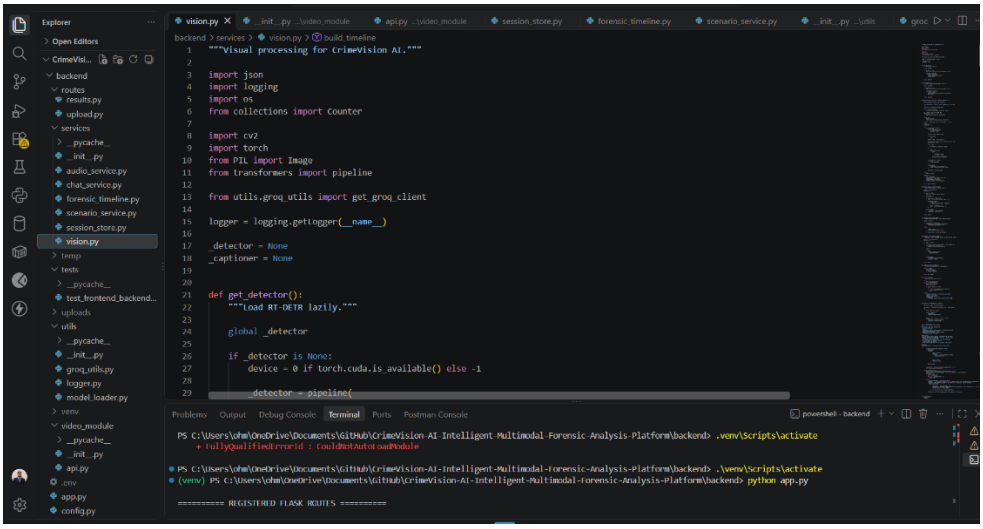
========== REGISTERED FLASK ROUTES ==========
Map([<Rule '/api/health' (OPTIONS, HEAD, GET) -> processing.processing_health>,
 <Rule '/api/image/analyze' (OPTIONS, POST) -> processing.analyze_image>,
 <Rule '/api/audio/analyze' (OPTIONS, POST) -> processing.analyze_audio>,
 <Rule '/api/video/analyze' (OPTIONS, POST) -> processing.analyze_video>,
 <Rule '/api/upload' (OPTIONS, HEAD, GET) -> upload.upload_status>,
 <Rule '/api/upload' (OPTIONS, POST) -> upload.upload_file>,
 <Rule '/api/chat' (OPTIONS, POST) -> chat.general_chat>,
 <Rule '/api/chat/<mode>' (OPTIONS, POST) -> chat.mode_chat>,
 <Rule '/api/chat' (OPTIONS, HEAD, GET) -> chat.chat_status>,
 <Rule '/api/audio' (OPTIONS, HEAD, GET) -> audio_analysis.audio_status>
```

6B. Milestone 2: Image Analysis Module Development

Milestone objective: implement the described capability and capture evidence of its behavior in the actual application.

Activity 2.1: Implement object detection for people, weapons, and evidence

Validate and decode the image, preprocess it as required by the model, run the configured detector, and convert outputs into a consistent schema containing label, bounding box, confidence, and source reference where available. Draw annotated boxes on a copy rather than overwriting the original. The detector can only identify classes supported by its trained weights.



Activity 2.2: Generate scene descriptions and possible scenario hypotheses

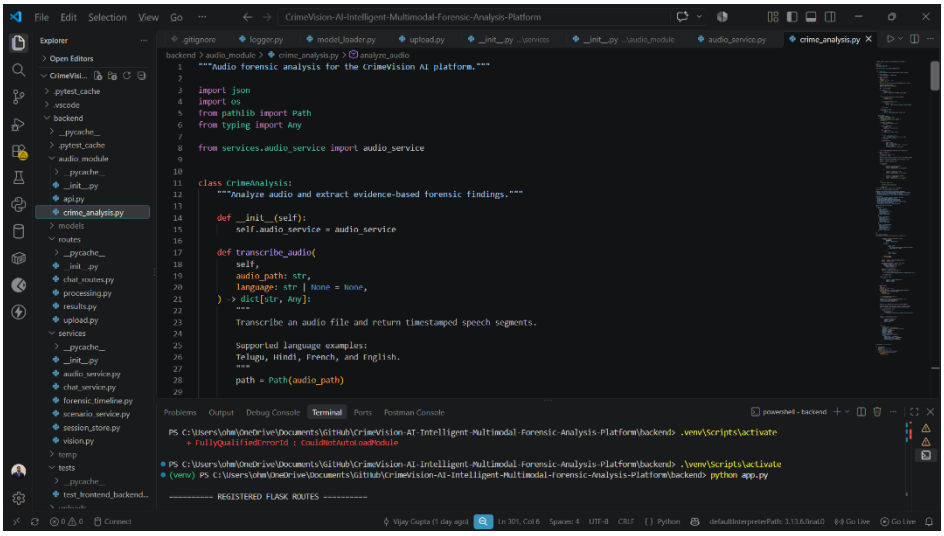
Use the configured captioning model (for example, BLIP, GIT, or a Florence alternative) to describe visible content. If hypothesis generation is enabled, provide only relevant model outputs to the reasoning step and label the result as tentative. Show the description alongside the original image so users can verify it.

6C. Milestone 3: Audio Analysis Module Development

Milestone objective: implement the described capability and capture evidence of its behavior in the actual application.

Activity 3.1: Implement audio transcription for witness statements

Validate and decode the audio file, pass it to Whisper, and capture the returned transcript and segment timestamps where available. Display the text for review and note that background noise, overlapping speakers, accents, and recording quality can affect accuracy.



Activity 3.2: Extract key events and generate structured timelines

Define a schema for event time, description, source transcript segment, and review status. Extract candidate events, normalize available timestamps, mark unknown times rather than guessing, sort events chronologically, and flag ambiguity or conflicts.

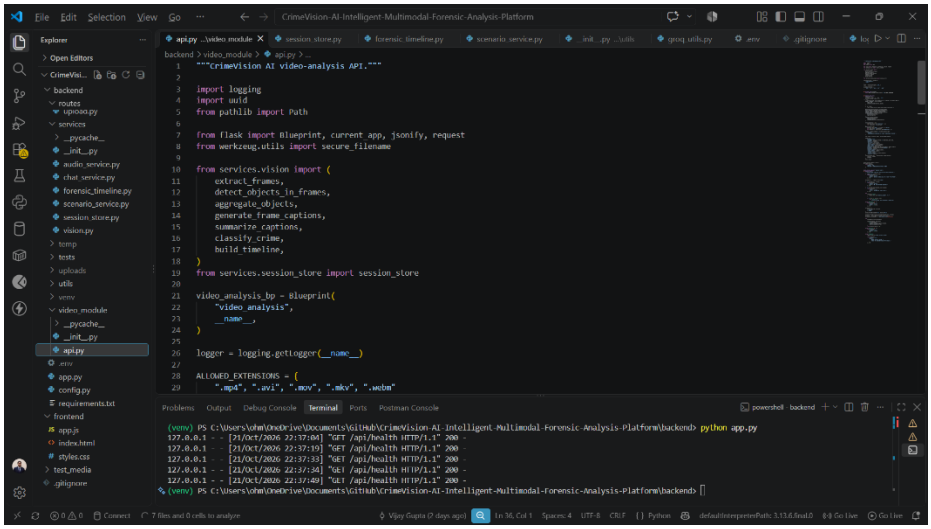
6D. Milestone 4: Video Analysis and Timeline Generation

Milestone objective: implement the described capability and capture evidence of its behaviour in the actual application.

Activity 4.1: Process video frames for object detection and action recognition

Read video metadata, choose and document a frame-sampling strategy, and run selected frames through the detector. Use a separate action-recognition model if action labels are

required; object detection alone does not prove an action. Attach observations to timestamps and handle decoding errors.



Activity 4.2: Generate second-by-second event timelines and crime type classification

Convert frame indices to timestamps, group related observations only when the rules support grouping, link each event to relevant frames, and run a configured classifier if available. Clearly mark crime-type labels as model predictions and expose uncertainty; do not make legal determinations.

6E. Milestone 5: Multimodal Fusion and Intelligence Generation

Milestone objective: implement the described capability and capture evidence of its behaviour in the actual application.

Activity 5.1: Fuse image, audio, and video outputs into a unified forensic timeline

Map outputs into a common schema that preserves modality, source filename, timestamp/frame reference, description, label, and confidence where available. Sort time-based entries and group related items only where justified. Retain provenance and identify missing or incompatible timestamps.

Activity 5.2: Implement natural-language scene querying and hypothesis generation

Assemble the stored analysis context and send the user's question with relevant context to the Groq LLM API. Prompt the model to answer only from that context, refer to supporting

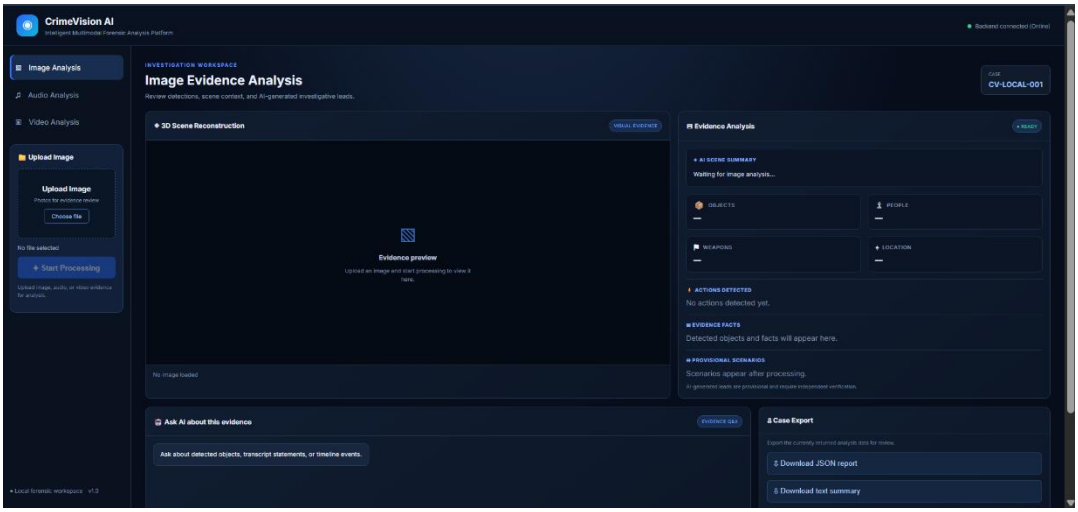
observations where possible, and state when information is insufficient. Test contradictory context, unsupported questions, and API failures.

6F. Milestone 6: Interface, Testing, and Deployment

Milestone objective: implement the described capability and capture evidence of its behaviour in the actual application.

Activity 6.1: Integrate all analysis modules into the CrimeVision AI dashboard

Provide navigation for implemented modules, connect forms to Flask endpoints, and show upload status, progress, validation errors, and results consistently. Display source references and timestamps where available and avoid exposing sensitive media unintentionally.



Activity 6.2: Validate system outputs using unit and integration tests

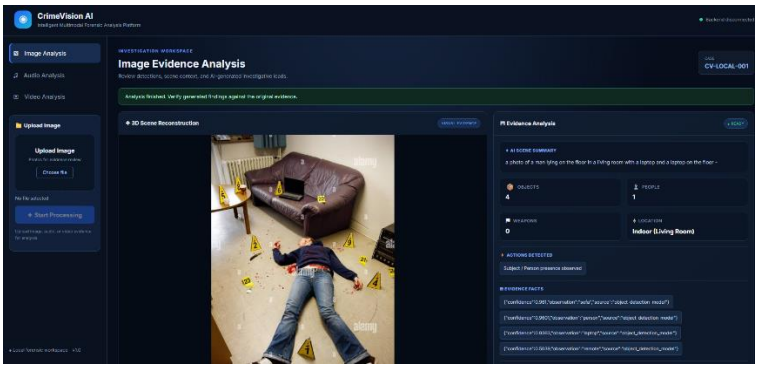
Unit-test validation, preprocessing, timestamp conversion, schema formatting, and report generation. Integration-test each full workflow and test missing files, unsupported formats, empty detections, model failures, API timeouts, and oversized uploads. Record expected and actual outcomes; do not claim success without test evidence.

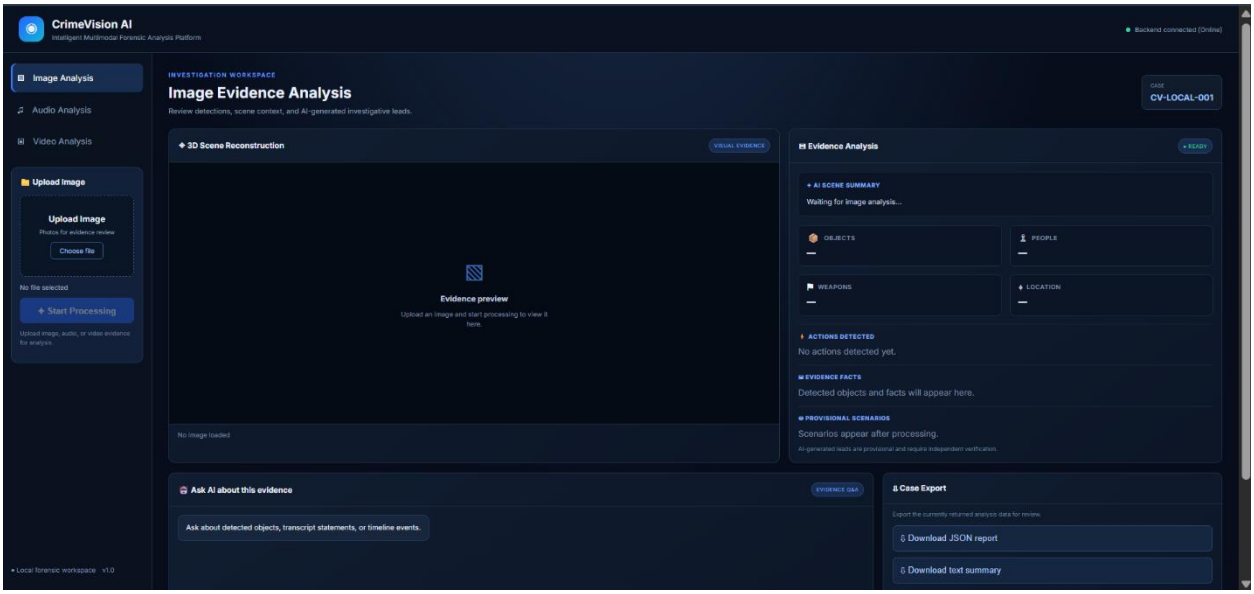
Activity 6.3: Generate forensic reports and finalize application deployment

Generate a report containing selected outputs, source filenames, processing date, timeline, and limitations. Verify the report against the dashboard. Configure production settings, secrets, logging, upload limits, and access controls; perform a deployment smoke test and document operational steps.

7. Exploring the Website: Web Pages, Input Sections, and Results

Use this section as a user guide. Update page names and fields to match the running application. The screenshots should be captured from the actual website rather than recreated from the project plan.





7. Conclusion

CrimeVision AI brings together computer-vision, audio-transcription, video-analysis, and language-model components to support structured review of digital evidence. The six milestones provide a logical development sequence: prepare the environment, build image and audio pipelines, process video and timelines, fuse multimodal outputs, and integrate, test, report, and deploy the application.

The platform's usefulness depends on traceability, transparent uncertainty, secure evidence handling, and careful validation as much as on model capability. Detections, captions, transcripts, event groupings, and hypotheses can be incomplete or wrong. The application

should help people review evidence efficiently while leaving consequential judgments to qualified human reviewers.

Before final submission, replace every screenshot placeholder with genuine captures from the completed application, verify the exact technology versions and interface fields, record actual test results, and revise any description that does not match the final code or deployment.